

Inductive Graph Neural Network with Causal Sampling for IoT Network Intrusion Detection System

Satriawan Rasyid Purnama

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

satriawanrasyidpurnama@mail.ugm.ac.id

Jazi Eko Istiyanto

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

jazi@ugm.ac.id

Muhammad Alfian Amrizal

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

muhammad.alfian.amrizal@ugm.ac.id

Vian Handika

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

vian.handika@mail.ugm.ac.id

Syafiqur Rochman

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

syafiqurrochman@mail.ugm.ac.id

Andi Dharmawan

Department of Computer Science and
Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

andi_dharmawan@mail.ugm.ac.id

Abstract— The widespread use of Internet of Things (IoT) devices causes many vulnerabilities. Today's network intrusion detection techniques require a data-driven approach to deal with many variations of new attacks daily. The machine learning method performs well in anomaly-based intrusion detection systems on network traffic. Recently, traditional machine learning processes network flow data without considering the network topology. Graph-based learning is capable of processing the topology of the data. An inductive Graph Neural Network is used to process large graphs. However, sampling is very challenging in aggregating relevant information from the neighborhood, which causes frequent noise. We propose the application of causal sampling to the Inductive Graph Neural Network model, E-GraphSAGE, to obtain relevant neighboring edges according to the causal weights. Our method was evaluated on publicly available datasets ToN-IoT with improved performance over random sampling, both with and without perturbation.

Keywords—graph neural network, intrusion detection system, causal sampling

I. INTRODUCTION

The rapid development of the Internet of Things (IoT), such as the emergence of voice assistant technology, sensors, automation tools, smart homes, and smart transportation, impacts and engages many industrial sectors and daily life [1]. Therefore, the issue of vulnerability and data security from both the producer and consumer sides is a significant concern [2]. IoT security improvements are carried out to prevent various cyberattacks that are still evolving [3]. Network Intrusion Detection System (NIDS) is a solution as a solid defense layer. However, the challenge is the capability to recognize new attacks.

Machine learning has emerged as a new paradigm to solve the problem of network intrusion detection. Machine learning has the advantage of especially those based on neural network deep learning that can operate end-to-end and practically to get optimal results [4]. However, traditional machine learning algorithms (based on non-graphs) today have not fully utilized the structural and topology information inherent in network flow data to detect complex network attacks, such as

in communication network problems [5]. Detecting complex network attacks with interrelated data structures is one of Graph Neural Networks' main strengths.

Graph-based learning for large graph processing requires expensive costs and computational time, so permutation invariant operations are needed. Therefore, GraphSAGE can run with a much faster computational process by applying the sampling and aggregation mechanism and does not depend too much on a fixed graph structure [6]. GraphSAGE is used for node embedding, but Lo et al. [7] changed it to edge embedding to tackle the NIDS problem, hence its name E-GraphSAGE. The main problem of GraphSAGE has been focused on sampling mechanisms. Recent solutions are carried out using data-driven learning [8], Graph Convolutional Networks (GCN) [9], and Graph Attention Network (GAT) [10] to improve classification performance on GraphSAGE, but all require high computational time. Zhang et al. [11] discovered causal weights between the neighbors of the target node and the labels using a causal inference and then applied those weights to the initial GraphSAGE sampling to achieve the best result. This causal sampling method offers better performance improvements, and its computation time is not much different from the original method of GraphSAGE.

At the same time, a robust model against noise, data distribution shift, and other perturbations are needed. Unstable detection performance is often caused by the nature of the data in the real world, which is often different in distribution from the training data [12]. Noise and changes over time due to infrastructure are also a problem. Causal bootstrapping [13] is often used to overcome this problem by resampling data according to the causal relationships between variables. The advantage of causal inference is to enable the machine learning algorithm to make causally robust predictions. In experiments, the perturbing technique is often used to train or test models under certain conditions of perturbation on the data, for example by perturbing some features [14], inject new samples in the training data [15], generating adversarial data [16], and generating adversarial attack [17].

Causal sampling is needed to solve the NIDS problem because the to noise that causes a decrease in performance of

model [18]. Therefore we propose the application of causal sampling to the E-GraphSAGE inductive learning model to reduce sample edges that may cause noise. Our main contribution is to use causal sampling rather than uniform random sampling to deal with any perturbation in NIDS data. Our method was tested on data without perturbation and with a certain perturbation ratio.

II. MATERIALS AND METHODS

A. Dataset

For this research, we selected two intrusion datasets. The characteristics features of the dataset are described in Table I. There are several categorical and numerical variables in the dataset. The target encoding technique is used to encode categorical variables then standard scaler applied to the data. The dataset used has an imbalanced data class problem. For the ToN-IoT [19] dataset, we use sample data from the train/test file that is already available on the web page.

TABLE I. OVERVIEW DATASETS FOR EXPERIMENT

Features	Variable Type	Unique Values
src_ip_port	categorical	460321
dst_ip_port	categorical	13147
proto	categorical	3
service	categorical	10
duration	numerical	124727
src_bytes	numerical	3056
dst_bytes	numerical	3511
conn_state	categorical	13
missed_bytes	numerical	929
src_pkts	numerical	414
src_ip_bytes	numerical	4549
dst_pkts	numerical	332
dst_ip_bytes	numerical	4577
dns_qclass	numerical	3
dns_qtype	numerical	12
dns_rcode	numerical	5
dns_AA	categorical	3
dns_RD	categorical	3
dns_RA	categorical	3
dns_rejected	categorical	3
ssl_version	categorical	4
ssl_cipher	categorical	6
ssl_resumed	categorical	3
ssl_established	categorical	3
http_trans_depth	categorical	11
http_method	categorical	4
http_version	categorical	2
http_request_body_len	numerical	6
http_response_body_len	numerical	68
http_status_code	numerical	8
http_orig_mime_types	categorical	3
http_resp_mime_types	categorical	10
label	categorical	10

Table II shows the distribution of each class on the dataset. We observe that these datasets had multiclass problems with one majority class (normal class) and several minority classes (different attacks class). The normal class has the highest distribution in each dataset, starting from 65.07% in the ToN-IoT dataset. Likewise, the attack class that has the smallest distribution in each dataset ranges from 0.22% in MITM attacks recorded by ToN-IoT. Imbalanced dataset characteristics can provide challenging scenarios for our research.

TABLE II. DISTRIBUTION ToN-IoT DATASETS CLASS

Class	Instance (size (%))
Normal	300,000 (65.07%)
Backdoor	20,000 (4.34%)
DDoS	20,000 (4.34%)
DoS	20,000 (4.34%)
Injection	20,000 (4.34%)
MIMT	1,295 (0.22%)
Ransomware	20,000 (4.34%)
Password	20,000 (4.34%)
Scanning	20,000 (4.34%)
XSS	20,000 (4.34%)

B. NIDS based on Edge Classification

Network flow data can be broken down into two components: identifiers and dense flow features. The identifier includes the addresses of the source and destination (IP and port combined). To avoid this type of attack being carried out on similar IP sources, we randomly replace all IP sources based on the range 172.16.0.1 to 172.31.0.1. The flow features include duration, transaction bytes, size of transmitted packets, etc. The source and destination identifiers are nodes, flow features are edges with their label, and all data assemble into a large graph. This intrusion detection mechanism is an edge classification task.

Fig. 1. shows the E-GraphSAGE architecture with the network data flow. Each network flow is denoted by E, which has two nodes including S as the source and D as the destination, with each node connected to its neighbors via other edges. E-GraphSAGE runs with two hops. E-GraphSAGE processes the main edge which is symbolized by Ei. Sampling on the 1-hop takes eight of ED and eight of ES, and sampling on the 2-hop takes eight edges each of the nodes connected to the ED and ES respectively. With the excluding features of src_ip_port, dst_ip_port, and label, the number of features used as edge features is 36. The neighbors of each hop are aggregated and concatenated with the previous hop and then multiplied by the matrix W. The dimensions of W on 1-hop it is 128, and on 2-hop it is 256. The last layer generates edge embedding, each concate produces a number of 512 dimensions, then the final fully-connected layer (logits) provides edge label prediction.

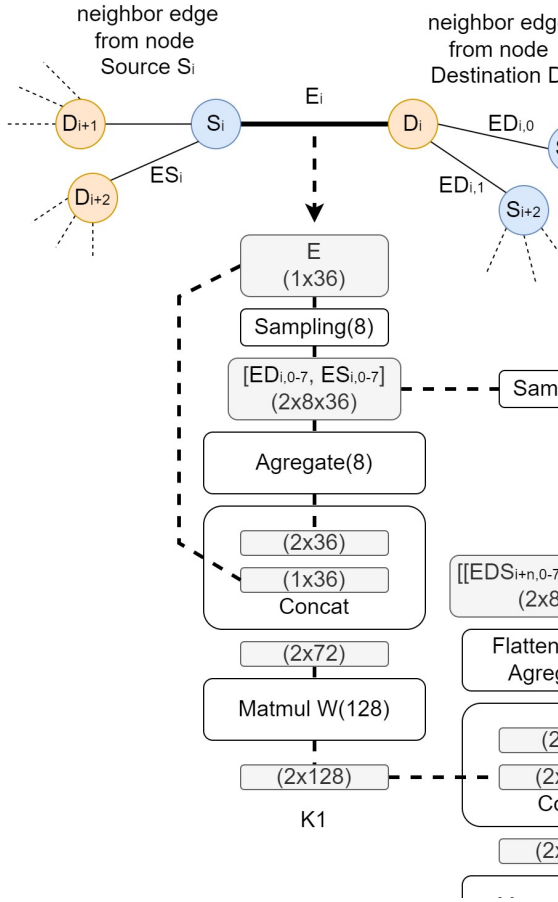


Fig. 1. E-GraphSAGE architecture with the network data flow

C. E-GraphSAGE

GraphSAGE is one of GNN's inductive learning models that utilize node features to study the distribution of features in a particular node's on-premises environment and its topology structure. GraphSAGE learns with a set of functions by gathering knowledge from the node environment. This learned aggregation function is applied during inference time to generate embedding for the nodes [6].

GraphSAGE uses convolutional graph layers, which describe the number of hops in which node information is aggregated for each iteration. The selection of aggregation functions $AGG_k, \forall k \in \{1, \dots, K\}$ is also important in combining selected node information. The algorithm works with the forward propagation stage and the backward propagation stage. At each iteration, the initialization of the neighbor is sampled to aggregate its information into a single vector. On the k layer, the information aggregated on the nodes v , based on neighboring samples $N(v)$, is expressed in Formula (1).

$$h_{N(v)}^k = AGG_k(\{h_u^{k-1}, \forall u \in N(v)\}) \quad (1)$$

Here, h_u^{k-1} it represents the embedding of the node u in the previous layer. The embedding of the entire node u on $N(v)$ is aggregated into the embedding of the node v on the layer k . Aggregation can use the averaging, maximum, and multiple steps of a complex neural network.

Then $h_{N(v)}^k$ contended with the embedding of the nodes in the previous layer and multiplied by the weight matrix W^k ,

and to obtain the result, it is necessary to go through a nonlinear activation function σ e.g. ReLU. This operation is called a transformation for each embedding of a node v on the layer k expressed in Formula (2).

$$h_v^k = \sigma(W^k \cdot \text{CONCAT}(h_v^{k-1}, h_{N(v)}^k)) \quad (2)$$

The final representation of the node v as z_v where is the embedding in the final layer is K expressed in Formula (3).

$$z_v = h_v^K, \quad \forall v \in V \quad (3)$$

In the case of IoT network flow data handling, the features available are not on the node but at the edge. Flow data is a feature of the edge. Thus, it is necessary to change the embedding done on the node to edge embedding so that the architecture for the IoT network flow data case is called E-GraphSAGE [7]. There are edge features $\{e_{uv}, \forall uv \in E\}$ as its input, and the vector x_v is initialized as a node feature with its number of dimensions equal to the number of edge features. The aggregation of nodes that originally used Formula (1), was changed to the edge aggregation expressed in Formula (4).

$$h_{N(v)}^k = AGG_k(\{e_{uv}^{k-1}, \forall u \in N(v), uv \in E\}) \quad (4)$$

Here, e_{uv}^{k-1} it represents the edge embedding uv of the node v on the layer $k-1$. The set represents $\{\forall u \in N(v), uv \in E\}$ a selected edge in neighborliness $N(v)$. Meanwhile, the transformation calculation still uses Formula (2). Whereas the embedding at the end layer K , any $z_v = h_v^K$, from edge uv to z_{uv}^K , is calculated as the conjecture of the embedding of nodes v and nodes u expressed in Formula (5).

$$z_{uv}^K = \text{CONCAT}(z_u^K, z_v^K), uv \in E \quad (4)$$

As for the learning of this graph using the backward propagation method. Gradient descent (Adam optimizer) is used for backward propagation to optimize parameters and parameters in the aggregation function W^k (if using an aggregation function that contains parameters). The classification loss function of each class M on the softmax activation function can be expressed in Formula (6).

$$L = -\sum_{i=1}^N \sum_{c=1}^M y_{ic} \log \left(\frac{\exp(z_{ic}^K)}{\sum_{c=1}^M \exp(z_{ic}^K)} \right) \quad (5)$$

D. Causal Sampling

Causal inference is a tool for modeling and processing data to identify interventional or counterfactual relationships of graph models [11]. In causal inference, the calculus obtains the intervention distribution, which states the probability that events will occur if the intervention is applied uniformly to a sample population, which is the basic method of representing the physical intervention with its effect $P(Y=y|do(X=x))(Y=y)(X=x)$. There is a probabilistic network.

Within GraphSAGE, causal sampling contributes to weighting neighboring nodes based on their causal weights

and selecting them based on those weights rather than randomly selecting them. Each node v_i it is used to predict the label y_i , which means y_i is the effect, and v_i is the cause. Then, the set $\{\forall u \in N(v)\}$ is also the cause of the effect y_i . Inside the causal graph, it U is defined as a causal path $v_i \leftarrow U \rightarrow y_i$. In such a way, $\{v_i, U\}$ it is also the cause of y_i . Here, U it can be thought of as the entire edge in the graph or determined by a certain ratio by a certain method such as the Bernoulli Distribution of the number of edges in the graph. However, if the arrow does not start from v_i then it is called the backdoor path, which is the vandal. The condition of connecting the causal path is that there is no descent against v_i the inside U , and U must omit all other causal paths including the back door path that is in between $v_i \rightarrow y_i$. To eliminate all other causal pathways within the U do calculus is used to identify the direct causal relationship between $v_i \rightarrow y_i$ and the causal bootstrap inference expressed in Formula (7).

$$p(y_i|do(v_i)) = \frac{1}{N} \sum_{n=1}^N K[y_i - y_n] \frac{K[v_i - v_n]}{\hat{p}(v_i|U)} \quad (7)$$

Where, $K[\cdot]$ is the kernel density estimate (KDE) obtained from the kernel Hilbert space (RKHS) reproducing function with the weight expressed in Formula (8), then Formula (7) becomes Formula (9). Meanwhile, it is $w_i \hat{p}(\cdot)$ the probability density function (PDF).

$$w_i = \frac{1}{N \hat{p}(v_i|U)} \quad (8)$$

$$p(y_i|do(v_i)) = w_i \sum_{n=1}^N K[y_i - y_n] \cdot K[v_i - v_n] \quad (9)$$

$$p(y_i|do(v_i)) = \begin{cases} w_i & y_i = y_n, v_i = v_n \\ 0 & \text{if not} \end{cases} \quad (10)$$

Formula (3.10) describes the fact that the intervention distribution of $v_i \rightarrow y_i$ the estimate is estimated by w_i the so-called causal weight. So that in the random sampling mechanism in GraphSAGE, it is replaced by sampling based on causal weights. This causal inference mechanism is effective for the sampling stage of the first order or 1-hop.

Implementation on E-GraphSAGE is done by taking all the neighbors in a 1-hop environment without including them in the training. For each $e_{uv_i}, \forall u \in N(v_i), uv \in E$ calculation $\hat{p}(e_{uv_i}, U)$ expressed in formula (11) and calculated $\hat{p}(U)$ in formula (12). It is then obtained $\hat{p}(e_{uv_i}|U)$ which is stated in Formula (13) as a PDF conditional to obtain causal weight w_i according to Formula (3.8). So that the entire sample of each edge e_{uv_i} for the node v_i has a weight w_i of. Those weights can be used as fixed samples for E-GraphSAGE training.

$$\hat{p}(e_{uv_i}, U) = \frac{1}{N} \sum_{n=1}^N K[e_{uv_i} - e_{uv_n}] \cdot K[U - U_n] \quad (11)$$

$$\hat{p}(U) = \frac{1}{N} \sum_{n=1}^N K[U - U_n] \quad (12)$$

$$\hat{p}(e_{uv_i}|U) = \frac{\hat{p}(e_{uv_i}, U)}{\hat{p}(U)} \quad (13)$$

E. Evaluation Measurement

Performance measurement in algorithm machine learning is commonly used to visualize the relationship between current and target predictions to evaluate the classifier's performance [2]. There is a measurement of performance, this study uses measurements for multi-class tasks. There $TP_i = p_{ii}, FP_i = \sum_{j=1, j \neq i}^r p_{ij}$, and $FN_i = \sum_{j=1, j \neq i}^r p_{ji}$, where p_{ij} is the result of predicting a class i as j for i and j is each r class. To search for Precision and Recall values is required $TP_i + FP_i = p_{i\cdot}$, and $TP_i + FN_i = p_{\cdot i}$. So that precision and recall measurements are carried out to get the macro-average F1-Score shown in Formula (14), Formula (15), and Formula (16), respectively.

$$precision_i = \frac{TP_i}{TP_i + FP_i} = \frac{p_{ii}}{p_{i\cdot}} \quad (14)$$

$$recall_i = \frac{TP_i}{TP_i + FN_i} = \frac{p_{ii}}{p_{\cdot i}} \quad (15)$$

$$macrof1 = \frac{2}{r} \times \sum_{i=1}^r \frac{p_{ii}}{p_{i\cdot} + p_{\cdot i}} \quad (16)$$

For multi-class tasks where TP_i, TN_i, FN_i , and FP_i each shows a True Positive class i , True Negative class i , False Negative class i , and False Positive class i for each class r class.

III. RESULT AND DISCUSSION

All models are implemented with the Pytorch from the Python library on google colab. The data splitting is done using a ratio of 5:2:3 for training, validation, and testing. The training was carried out using an Adam optimizer with a learning rate of 0.01 and cross-entropy loss with weighted class balancing. Training run among two epochs with batch size 500.

The model runs for each dataset individually, which consists of the stages of sampling, aggregation, and edge classification. The algorithm runs two hops. Causal sampling is done by selecting 32 random nodes and determining which one will be the edge sample candidate for each neighbor of the two connecting nodes. Then a maximum of 8 nodes are taken before being aggregated. Finally, the neighbors of the two connecting nodes will be concatenated and proceed with edge classification.

ToN-IoT datasets were evaluated with the f1-score macro metric for multi-class classification tasks. The scenario for the datasets is given perturbation with empty edges. The perturbation ratios of 10%-50% and 0% (without perturbation) given in train and test data randomly by Bernoulli distribution. From the results obtained, Table III. shows that the higher the perturbation ratio, the lower the performance. E-GraphSAGE using causal sampling is higher than the random sampling almost all over perturbation ratio. At 0% perturbation ratio, macro f1-score gets 85.24% for causal sampling and 84.36% for random sampling. At a perturbation ratio of 10%, the macro f1-score got 83.90% for causal sampling and 83.39% for random sampling. At 20% perturbation ratio, macro f1-score got 73.84% for causal sampling and 74.87% for random sampling. At 30% perturbation ratio, macro f1-score got 73.84% for causal sampling and 71.86% for random sampling. At 40%

perturbation ratio, macro f1-score got 72.80% for causal sampling and 71.39% for random sampling. At a perturbation ratio of 50%, the macro f1-score got 71.90% for causal sampling and 69.08% for random sampling. The result shows that the causal sampling method efficiently chooses a representative edge for the NIDS problem.

TABLE III. CLASSIFICATION PERFORMANCE ON ToN-IoT TEST SET

Perturbation Ratio	macroF1-Score	
	Random Sampling	Causal Sampling
0	0.8436	0.8524
0.1	0.8339	0.8390
0.2	0.7487	0.7384
0.3	0.7186	0.7318
0.4	0.7139	0.7280
0.5	0.6908	0.7190

E-GraphSAGE with causal sampling has better resilience than random sampling in terms of the distribution of the average f1-score in each class. Fig. 2. shows that the classes that have a high f1-score are almost similar between the two, such as Normal, Password, and Backdoor. However, some classes including Scanning, DoS, Injection, and MIMT are slightly higher in causal sampling. However, for XSS and Ransomware causal sampling is slightly below random sampling. The comparison of scores on DDos is much better when using causal sampling, which means a significant decrease occurs when using random sampling when various perturbations are encountered. Robust model against noise will tend to give the effect of improving model performance on imbalanced class problems [20].

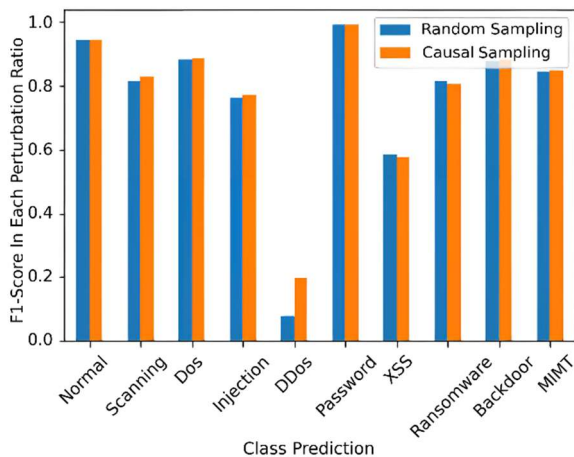


Fig. 2. The distribution of the average f1-score for each class on the ToN-IoT test data for causal sampling and random sampling

IV. CONCLUSION

In this paper, we propose the application of causal sampling to the E-GraphSAGE model for the NIDS problem. The results of our experiments for ToN-IoT datasets are that the effect of causal sampling improves performance with the best f1-score-macro 85.24% for ToN-IoT with no perturbation. When given perturbation, it is always superior to random sampling. Although this method shows better robustness, the improvement in classification performance is

limited, and the higher the disturbance, the smaller the number of neighbors selected. Therefore, future work will study some interesting things about perturbation techniques and their types to further investigate the effect on causal sampling. In addition, discussing the advantages and limitations of the causality technique for NIDS problems more comprehensively, and comprehensive analysis in terms of its attack mechanism to better model graphs on IoT networks is interesting for future research.

ACKNOWLEDGMENT

This research has been funded by Universitas Gadjah Mada via the RTA batch II scheme number 5722/UN1.P.III/Dit-Lit/PT.01.05/2022. The authors are grateful for the financial support.

REFERENCES

- [1] Huy-Trung Nguyen, Quoc-Dung Ngo, and Van Hoang Le, "A novel graph-based approach for IoT botnet detection," *International Journal of Information Security*, vol. 19, no. 5, pp. 567–577, 2020.
- [2] A. Kumar, M. Shridhar, S. Swaminathan, and T. J. Lim, "Machine learning-based early detection of IoT botnets using network-edge traffic," *Computers and Security*, vol. 117, 2022, doi: 10.1016/j.cose.2022.102693.
- [3] S. Roy, J. Li, B. J. Choi, and Y. Bai, "A lightweight supervised intrusion detection mechanism for IoT networks," *Future Generation Computer Systems*, vol. 127, pp. 276–285, 2022, doi: 10.1016/j.future.2021.09.027.
- [4] W. Jiang, "Graph-based deep learning for communication networks: A survey," *Computer Communications*, vol. 185, pp. 40–54, 2022, doi: 10.1016/j.comcom.2021.12.015.
- [5] J. Suárez-Varela et al., "Graph Neural Networks for Communication Networks: Context, Use Cases and Opportunities," 2021.
- [6] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in Neural Information Processing Systems*, vol. 2017-Decem, pp. 1025–1035, 2017.
- [7] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A Graph Neural Network based Intrusion Detection System for IoT," in *noms*, 2022.
- [8] J. Oh, K. Cho, and J. Bruna, "Advancing GraphSAGE with A Data-Driven Node Sampling," 2019.
- [9] S. Abu-El-Haija, A. Kapoor, B. Perozzi, and J. Lee, "N-GCN: Multi-scale graph convolution for semi-supervised node classification," *35th Conference on Uncertainty in Artificial Intelligence*, UAI 2019, 2019.
- [10] P. Veličković, A. Casanova, P. Liò, G. Cucurull, A. Romero, and Y. Bengio, "Graph attention networks," *6th International Conference on Learning Representations, ICLR 2018 - Conference Track Proceedings*, vol. 1050, p. 20, 2018, doi: 10.1007/978-3-031-01587-8_7.
- [11] T. Zhang, H. R. Shan, and M. A. Little, "Causal GraphSAGE: A robust graph method for classification based on causal sampling," *Pattern Recognition*, vol. 128, 2022, doi: 10.1016/j.patcog.2022.108696.
- [12] Z. R. Zeng, W. Peng, and D. Zeng, "Improving the Stability of Intrusion Detection with Causal Deep Learning," *IEEE Transactions on Network and Service Management*, 2022, doi: 10.1109/TNSM.2022.3193099.
- [13] M. A. Little and R. Badawy, "Causal bootstrapping," 2019.
- [14] J. Aiken and S. Scott-Hayward, "Investigating Adversarial Attacks against Network Intrusion Detection Systems in SDNs," *IEEE Conference on Network Function Virtualization and Software Defined Networks, NFV-SDN 2019 - Proceedings*, 2019, doi: 10.1109/NFV-SDN47374.2019.9040101.
- [15] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, "Modeling Realistic Adversarial Attacks against Network Intrusion Detection Systems," *Digital Threats: Research and Practice*, 2021, doi: 10.1145/3469659.
- [16] E. Alhajjar, P. Maxwell, and N. Bastian, "Adversarial machine learning in Network Intrusion Detection Systems," *Expert Systems with Applications*, vol. 186, 2021, doi: 10.1016/j.eswa.2021.115782.
- [17] N. Wang, Y. Chen, Y. Xiao, Y. Hu, W. Lou, and T. Hou, "MANDA: On Adversarial Example Detection for Network Intrusion Detection

- System,” IEEE Transactions on Dependable and Secure Computing, 2022, doi: 10.1109/TDSC.2022.3148990.
- [18] D. Li, D. Kotani, and Y. Okabe, “Improving Attack Detection Performance in NIDS Using GAN,” Proceedings - 2020 IEEE 44th Annual Computers, Software, and Applications Conference, COMPSAC 2020, pp. 817–825, 2020, doi: 10.1109/COMPSAC48688.2020.0-162.
- [19] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and Adna N Anwar, “TON-IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems,” IEEE Access, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [20] R. K. L. Kennedy, J. M. Johnson, and T. M. Khoshgoftaar, “The Effects of Class Label Noise on Highly-Imbalanced Big Data,” Proceedings - International Conference on Tools with Artificial Intelligence, ICTAI, vol. 2021-November, pp. 1427–1433, 2021, doi: 10.1109/ICTAI52525.2021.00227.